Arquitetura - Aula 2

Desenvolvimento Dispositívos Móveis II

Jefferson Rodrigo Speck

Material didático produzido para apoio às aulas.

Sumário

1 Arquitetura em Camadas no Flutter **2**
1.1 Estrutura típica de pastas em projetos Flutter . . . 3

2 Camada Presentation **3**
2.1 Responsabilidades . . . 4
2.2 Restrições arquiteturais . . . 4
2.3 Exemplo de implementação de página . . . 5

3 ViewModels **6**
3.1 Estrutura típica . . . 6
3.2 Representação do estado da tela . . . 6
3.3 Coordenação de ações . . . 7
3.4 Responsabilidades arquiteturais . . . 8
3.5 Restrições arquiteturais . . . 8

4 Camada Domain **8**
4.1 Entidades . . . 9
4.2 Repositórios . . . 9
4.3 Independência do domínio . . . 10

5 Camada Data **11**
5.1 Models (DTO) . . . 11
5.2 Datasources . . . 12
5.3 Implementação do repositório . . . 13
5.4 Responsabilidades arquiteturais . . . 14

6 Camada Core **14**
6.1 Tratamento de erros . . . 14
6.2 Cliente HTTP . . . 15
6.3 Responsabilidades arquiteturais . . . 15

7 Fluxo de Execução **16**

8 Dependência entre Camadas **17**

9 Exercícios **19**
9.1 Atividade Guiada: Construção de uma Aplicação Flutter com Arquitetura em Camadas . . . 19
9.2 Atividade Guiada 2: Refatoração Arquitetural . . . 24

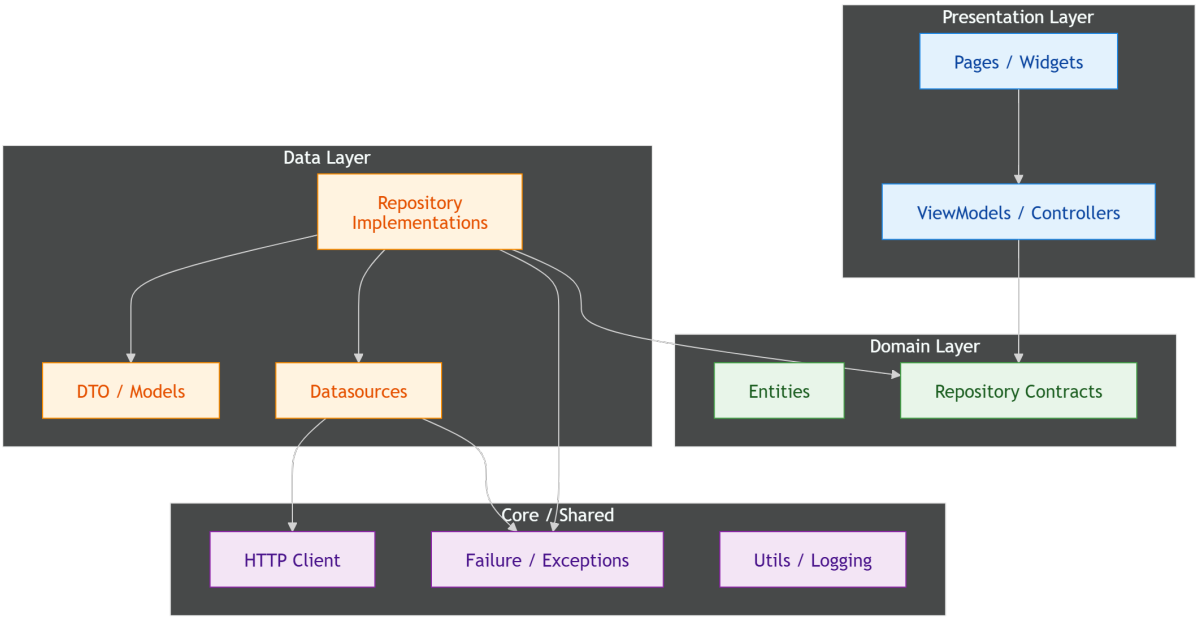
1 Arquitetura em Camadas no Flutter

Neste material será apresentada uma organização arquitetural amplamente utilizada no desenvolvimento de aplicações Flutter baseada na separação de responsabilidades entre camadas (*layered architecture*). Essa abordagem busca estruturar o sistema de forma que diferentes aspectos da aplicação — interface, regras de negócio e acesso a dados — sejam tratados em módulos distintos e bem definidos.

A motivação central dessa organização é reduzir acoplamento entre componentes, favorecer a manutenção do código ao longo do tempo e permitir maior testabilidade das regras de negócio. Em aplicações móveis modernas, nas quais a interface de usuário evolui rapidamente e integra-se frequentemente com diferentes serviços externos, como APIs, bancos de dados, sistemas de autenticação, entre outros, a ausência de uma arquitetura clara tende a gerar sistemas altamente acoplados e difíceis de evoluir.

A Figura 1 ilustra de forma geral a organização em camadas dentro de um projeto Flutter.

Figura 1: Arquitetura em camadas para aplicações Flutter



Fonte: Elaborado pelo autor.

Nesse contexto, a arquitetura adotada organiza o projeto em quatro grandes áreas, cada uma com responsabilidades específicas:

- **presentation**: responsável pela interface com o usuário. Essa camada contém páginas, widgets e componentes visuais da aplicação. Sua função principal é apresentar dados e capturar eventos do usuário, delegando o processamento das ações às camadas inferiores. Idealmente, essa camada não contém regras de negócio nem lógica de acesso a dados.

- **domain**: representa o núcleo conceitual do sistema. Aqui estão definidas as entidades do domínio e os contratos (*interfaces*) que descrevem as operações necessárias para manipular essas entidades. Essa camada é independente de *frameworks*, bibliotecas externas ou detalhes de infraestrutura, concentrando exclusivamente as regras de negócio.

- **data**: responsável pela implementação concreta do acesso aos dados. Essa camada contém repositórios, modelos de transferência de dados (DTOs) e componentes responsáveis pela comunicação com APIs, bancos de dados ou sistemas externos. Em outras palavras, ela traduz as necessidades definidas no domínio em operações técnicas de acesso a dados.

- **core**: concentra elementos de infraestrutura compartilhados por toda a aplicação, como clientes HTTP, tratamento de erros, utilitários, serviços de logging ou mecanismos de configuração. Esses componentes não pertencem diretamente ao domínio do problema, mas são necessários para o funcionamento da aplicação.

Essa separação em camadas estabelece também uma hierarquia de dependências. Em geral, a camada de apresentação depende do domínio para executar operações do sistema, enquanto a camada de dados implementa os contratos definidos no domínio. O domínio, por sua vez, não deve depender de nenhuma outra camada, preservando sua independência conceitual e permitindo que as regras de negócio sejam testadas isoladamente, onde cada camada possui responsabilidades bem definidas e dependências controladas.

1.1 Estrutura típica de pastas em projetos Flutter

A organização arquitetural descrita anteriormente pode ser refletida diretamente na estrutura de diretórios do projeto. Uma organização comum em aplicações Flutter baseadas em separação de camadas é apresentada a seguir.

```
lib/
  presentation/
    pages/
    widgets/
    viewmodels/

  domain/
    entities/
    repositories/

  data/
    models/
    datasources/
    repositories/

  core/
    errors/
    network/
    utils/
```

Cada uma dessas pastas corresponde a uma responsabilidade arquitetural específica dentro da aplicação.

Regra de dependência arquitetural:
presentation → domain
data → domain
domain → nenhuma camada

2 Camada Presentation

A camada de apresentação constitui o ponto de contato entre o sistema e o usuário. Ela é responsável por traduzir o estado interno da aplicação em elementos visuais compreensíveis e por capturar as interações realizadas pelo usuário na interface.

Em aplicações Flutter, essa camada é tipicamente composta por páginas (*pages*), widgets reutilizáveis e componentes responsáveis pela organização da interface. Seu papel central é transformar dados provenientes das camadas inferiores em representações visuais, permitindo que o usuário compreenda o estado atual da aplicação e execute ações sobre ele.

Uma característica fundamental dessa camada é a ausência de regras de negócio. Toda lógica relacionada à manipulação de dados, validação de regras ou acesso a serviços externos deve ser delegada a componentes de coordenação, como *viewmodels* ou controladores presentes na mesma camada, ou ainda às camadas inferiores da arquitetura.

Essa separação é essencial para manter a interface desacoplada da lógica do sistema. Quando a camada de apresentação passa a concentrar regras de negócio ou lógica de acesso a dados, o resultado tende a ser uma aplicação difícil de testar, difícil de manter e altamente dependente de detalhes específicos da interface.

2.1 Responsabilidades

Dentro de uma arquitetura bem estruturada, a camada de apresentação desempenha um conjunto específico de responsabilidades.

- **Renderização da interface**: consiste na construção da estrutura visual da aplicação a partir de widgets Flutter. A interface deve refletir o estado atual da aplicação, exibindo listas, formulários, mensagens de erro ou indicadores de carregamento conforme os dados disponíveis.
- **Observação de estado**: os componentes da interface observam objetos de estado produzidos por controladores ou *viewmodels*. Sempre que esse estado é atualizado, a interface é reconstruída automaticamente para refletir a nova situação da aplicação.
- **Captura de eventos do usuário**: interações como cliques, gestos, submissão de formulários ou navegação entre telas são capturadas pela interface e encaminhadas para componentes responsáveis pelo fluxo da aplicação.
- **Composição visual**: widgets menores são organizados para formar estruturas visuais mais complexas, promovendo reutilização de componentes e maior clareza na organização da interface.

2.2 Restrições arquiteturais

Para preservar a separação de responsabilidades, determinadas operações não devem ocorrer dentro da camada de apresentação.

- **Comunicação direta com serviços externos**: chamadas HTTP ou acesso a APIs externas devem ocorrer em camadas responsáveis por acesso a dados, evitando que a interface se torne dependente de detalhes de rede.
- **Manipulação de formatos de dados**: operações de serialização e desserialização de dados, como conversão entre JSON e objetos do sistema, devem ser tratadas na camada de dados.
- **Implementação de regras de negócio**: validações complexas ou decisões relacionadas ao domínio da aplicação devem permanecer na camada de domínio, preservando a independência conceitual do modelo de negócio.

2.3 Exemplo de implementação de página

O exemplo a seguir (Código 1) apresenta uma página Flutter responsável por exibir uma lista de produtos. A interface observa o estado produzido por um *viewmodel* e reage automaticamente às mudanças desse estado.

Código 1: Página responsável por exibir a lista de produtos

```
class ProductPage extends StatelessWidget {
  final ProductViewModel viewModel;
  const ProductPage({super.key, required this.viewModel});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Products")),
      body: ValueListenableBuilder<ProductState>(
        valueListenable: viewModel.state,
        builder: (context, state, _) {
          if (state.isLoading) {
            return const Center(
              child: CircularProgressIndicator()
            );
          }
          if (state.error != null) {
            return Center(
              child: Text(state.error!)
            );
          }
          return ListView.builder(
            itemCount: state.products.length,
            itemBuilder: (context, index) {
              final product = state.products[index];
              return ListTile(
                title: Text(product.name),
                subtitle: Text(product.price.toString()),
              );
            },
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: viewModel.loadProducts,
        child: const Icon(Icons.refresh),
      ),
    );
  }
}
```

Fonte: Elaborado pelo autor.

Nesse exemplo, a página não contém regras de negócio nem lógica de acesso a dados. Seu papel é observar o estado produzido pelo *viewmodel* e representar visualmente esse estado na interface, permitindo que o usuário interaja com o sistema de forma clara e responsiva.

3 ViewModels

Os *ViewModels* desempenham um papel central na coordenação do fluxo de interação entre a interface do usuário e as camadas responsáveis pelas regras de negócio. Enquanto a camada de apresentação concentra-se exclusivamente na construção da interface, os *ViewModels* atuam como mediadores que organizam o estado da aplicação e conduzem as ações solicitadas pelo usuário.

Em termos conceituais, o *ViewModel* representa a lógica de apresentação da aplicação. Ele não contém regras de negócio do domínio, mas é responsável por coordenar chamadas às camadas inferiores e transformar os resultados dessas operações em estados compreensíveis pela interface.

Essa abordagem permite que a interface permaneça simples e declarativa, enquanto o fluxo de dados e de ações é centralizado em componentes específicos, mais fáceis de testar e evoluir.

Uma implementação típica dessa camada é composta por duas estruturas principais: uma classe responsável por representar o estado da tela e outra responsável por gerenciar as ações que podem modificar esse estado.

3.1 Estrutura típica

Em projetos Flutter organizados dessa forma, é comum separar explicitamente o estado da lógica que manipula esse estado. Essa separação melhora a clareza arquitetural e facilita testes unitários.

- **Classe de estado**: representa a condição atual da interface em determinado momento. Ela contém apenas dados imutáveis que descrevem o estado da tela.
- **Classe de ViewModel**: coordena ações da aplicação, realiza chamadas ao domínio e produz novas versões do estado sempre que ocorre alguma alteração relevante.

Uma estrutura de arquivos comum é apresentada a seguir.

```
product_viewmodel.dart
product_state.dart
```

3.2 Representação do estado da tela

O estado da interface deve ser modelado como um objeto imutável. Isso significa que, em vez de alterar propriedades existentes, novas versões do estado são criadas sempre que ocorre uma atualização. Essa estratégia reduz efeitos colaterais, facilita a rastreabilidade das mudanças de estado e torna o comportamento da interface mais previsível.

No exemplo apresentado no Código 2, o estado da tela é representado por três elementos principais. Um indicador de carregamento permite identificar quando uma operação assíncrona está em execução. Uma lista de produtos representa os dados atualmente disponíveis na interface. Por fim, um campo de erro possibilita comunicar falhas ocorridas durante a execução de operações, permitindo que a interface apresente mensagens adequadas ao usuário.

Código 2: Estrutura de estado utilizada para representar a situação atual da interface

```
class ProductState {
  final bool isLoading;
  final List<Product> products;
  final String? error;

  const ProductState({
    this.isLoading = false,
    this.products = const [],
```

```
    this.error,
  });

  ProductState copyWith({
    bool? isLoading,
    List<Product>? products,
    String? error,
  }) {
    return ProductState(
      isLoading: isLoading ?? this.isLoading,
      products: products ?? this.products,
      error: error,
    );
  }
}
```

Fonte: Elaborado pelo autor.

A presença do método `copyWith` permite gerar novas versões do estado alterando apenas os campos necessários, preservando os demais valores.

3.3 Coordenação de ações

O Código 3 apresenta uma implementação de *ViewModel* responsável por coordenar o carregamento de produtos a partir do repositório. Esse componente atua como intermediário entre a interface e as camadas responsáveis pelo acesso a dados, garantindo que a lógica de fluxo da aplicação permaneça separada da construção da interface.

Durante a execução do método responsável pelo carregamento de dados, o estado da tela é atualizado para indicar o início da operação. Após a obtenção dos dados, um novo estado é produzido contendo a lista de produtos. Caso ocorra alguma falha durante a execução, o estado é atualizado com a mensagem de erro correspondente, permitindo que a interface apresente essa informação ao usuário.

Código 3: Implementação de ViewModel responsável por coordenar o carregamento de produtos

```
class ProductViewModel {
  final ProductRepository repository;

  final ValueNotifier<ProductState> state =
      ValueNotifier(const ProductState());

  ProductViewModel(this.repository);

  Future<void> loadProducts() async {
    state.value = state.value.copyWith(isLoading: true);
    try {
      final products = await repository.getAll();
      state.value = state.value.copyWith(
        isLoading: false,
        products: products,
      );
    } catch (e) {
      state.value = state.value.copyWith(
        isLoading: false,
        error: e.toString(),
      );
    }
```

```
  }
}
```

Fonte: Elaborado pelo autor.

Esse padrão cria um fluxo de dados previsível no qual o estado da interface é sempre atualizado a partir de ações explícitas.

3.4 Responsabilidades arquiteturais

Dentro dessa arquitetura, os *ViewModels* possuem responsabilidades bem definidas.

- **Gerenciamento de estado**: manter e atualizar o estado observado pela interface, garantindo que qualquer mudança seja refletida na renderização da tela.
- **Coordenação de operações**: encaminhar solicitações do usuário para as camadas responsáveis pelas regras de negócio ou acesso a dados.
- **Tratamento do fluxo da aplicação**: organizar sequências de ações, como iniciar carregamentos, tratar erros e atualizar o estado da interface após cada operação.

3.5 Restrições arquiteturais

Para preservar a separação de responsabilidades da arquitetura, determinadas operações não devem ocorrer dentro do *ViewModel*.

- **Manipulação de formatos de dados**: conversões entre JSON e objetos do sistema pertencem à camada de dados.
- **Comunicação direta com APIs**: chamadas HTTP devem ser encapsuladas em *datasources* ou repositórios.
- **Dependência de modelos externos**: objetos de transferência de dados utilizados por APIs não devem aparecer nessa camada.

Quando essas restrições são respeitadas, o *ViewModel* permanece focado na coordenação do fluxo da aplicação, mantendo a interface desacoplada de detalhes de infraestrutura.

4 Camada Domain

A camada de domínio representa o núcleo conceitual da aplicação. É nessa camada que são modelados os conceitos fundamentais do problema que o sistema busca resolver. Diferentemente das camadas responsáveis pela interface ou pela infraestrutura técnica, o domínio descreve a lógica essencial do sistema de forma independente de qualquer tecnologia específica.

Em termos arquiteturais, essa camada é responsável por encapsular as regras de negócio e os conceitos que caracterizam o problema tratado pela aplicação. Isso significa que classes presentes nessa camada não devem depender de frameworks de interface, bibliotecas de rede, mecanismos de persistência ou qualquer outro detalhe relacionado à infraestrutura do sistema.

Essa independência é um princípio fundamental em arquiteturas baseadas em separação de camadas. Ao manter o domínio isolado de aspectos técnicos da implementação, torna-se possível evoluir a interface da aplicação, substituir mecanismos de armazenamento ou modificar integrações externas sem comprometer as regras centrais do sistema.

Em aplicações Flutter organizadas segundo esse princípio, a camada de domínio normalmente contém dois tipos principais de elementos: entidades e contratos de repositório.

4.1 Entidades

Entidades representam os conceitos fundamentais do domínio do problema. Elas descrevem objetos que possuem identidade própria dentro do sistema e que carregam informações relevantes para o funcionamento da aplicação.

Diferentemente de estruturas voltadas para comunicação com APIs ou armazenamento em banco de dados, as entidades são modeladas com foco no significado conceitual dos dados e não em sua representação técnica.

Em arquiteturas bem estruturadas, entidades são frequentemente projetadas como objetos imutáveis. Isso significa que, após sua criação, os valores de suas propriedades não são modificados diretamente. Caso seja necessário alterar algum atributo, uma nova instância da entidade deve ser criada contendo os novos valores desejados.

Essa abordagem contribui para reduzir efeitos colaterais no sistema, facilita a escrita de testes e torna o comportamento da aplicação mais previsível, especialmente em cenários que envolvem programação assíncrona ou atualização frequente de estado.

O Código 4 apresenta um exemplo de entidade simples representando um produto no domínio da aplicação.

Código 4: Entidade de domínio representando um produto

```
class Product {
  final int id;
  final String name;
  final double price;

  const Product({
    required this.id,
    required this.name,
    required this.price,
  });
}
```

Fonte: Elaborado pelo autor.

Nesse exemplo, a classe `Product` representa uma entidade do domínio relacionada ao gerenciamento de produtos. A classe contém apenas os atributos necessários para representar esse conceito dentro da aplicação, sem depender de bibliotecas externas, formatos de serialização ou mecanismos de persistência. Essa simplicidade é intencional: entidades devem permanecer focadas na representação conceitual do domínio, evitando incluir detalhes técnicos que pertencem a outras camadas da arquitetura.

> Princípio da Inversão de Dependência
> Camadas de alto nível não devem depender de camadas de baixo nível. Ambas devem depender de abstrações.

4.2 Repositórios

Além das entidades, a camada de domínio também define contratos responsáveis por descrever como os dados podem ser acessados ou manipulados dentro do sistema. Esses contratos são chamados de repositórios. Um repositório não representa necessariamente um banco de dados ou uma API. Em vez disso, ele define um conjunto de operações que permitem recuperar ou modificar entidades do domínio. A implementação concreta dessas operações é delegada a camadas responsáveis pelo acesso a dados.

Essa abordagem segue o princípio da inversão de dependência, segundo o qual as camadas superiores da arquitetura devem depender de abstrações e não de implementações concretas. Nesse modelo, o domínio define contratos que descrevem quais operações precisam existir, enquanto as camadas inferiores fornecem implementações concretas para esses contratos.

O Código 5 apresenta um exemplo de contrato de repositório definido na camada de domínio. Esse contrato descreve operações relacionadas ao gerenciamento de produtos, permitindo recuperar, adicionar, atualizar e remover entidades do sistema sem expor detalhes sobre como esses dados são obtidos ou persistidos.

Código 5: Contrato de repositório definido na camada de domínio

```
abstract class ProductRepository {
  Future<List<Product>> getAll();

  Future<void> add(Product product);

  Future<void> update(Product product);

  Future<void> delete(int id);
}
```

Fonte: Elaborado pelo autor.

Nesse exemplo, o repositório define operações básicas relacionadas ao gerenciamento de produtos. A camada de domínio especifica quais operações são necessárias para manipular essas entidades, mas não define como essas operações serão realizadas. A implementação concreta dessas operações é responsabilidade da camada de dados, que pode utilizar diferentes mecanismos de acesso à informação, como APIs, bancos de dados locais ou serviços externos. Dessa forma, o domínio permanece independente de detalhes técnicos de infraestrutura.

A implementação concreta desse contrato pode utilizar diferentes mecanismos de acesso a dados, como APIs REST, bancos de dados locais ou serviços externos. Independentemente da tecnologia utilizada, a interface definida pelo repositório permanece a mesma.

4.3 Independência do domínio

Uma característica essencial dessa camada é sua independência em relação a detalhes técnicos da aplicação. Classes presentes no domínio não devem possuir dependências diretas de bibliotecas de interface, frameworks de rede ou mecanismos de persistência.

Essa independência oferece diversas vantagens arquiteturais.

- **Testabilidade**: regras de negócio podem ser testadas isoladamente, sem necessidade de inicializar componentes de interface ou infraestrutura.
- **Evolução tecnológica**: mudanças em frameworks, bibliotecas ou serviços externos não afetam diretamente o modelo conceitual do sistema.
- **Clareza conceitual**: a lógica central da aplicação permanece organizada em torno do problema de negócio, e não de detalhes técnicos de implementação.

Ao preservar essa separação, a camada de domínio torna-se o elemento mais estável da arquitetura, servindo como referência conceitual para todas as demais camadas do sistema.

5 Camada Data

A camada de dados é responsável por implementar o acesso efetivo às fontes de informação utilizadas pela aplicação. Enquanto o domínio define quais operações precisam existir por meio de contratos de repositório, a camada *data* fornece as implementações concretas que realizam essas operações utilizando mecanismos técnicos específicos.

Em termos arquiteturais, essa camada funciona como uma ponte entre o modelo conceitual do sistema e os detalhes de infraestrutura responsáveis pela obtenção e armazenamento de dados. Esses detalhes podem incluir comunicação com APIs REST, acesso a bancos de dados locais, leitura de arquivos ou integração com serviços externos.

Uma característica importante dessa camada é a separação entre três elementos principais: modelos de dados utilizados para comunicação externa, componentes responsáveis por operações de entrada e saída e implementações concretas dos repositórios definidos no domínio.

5.1 Models (DTO)

Modelos de dados, frequentemente chamados de *Data Transfer Objects* (DTO), são estruturas utilizadas para representar informações no formato exigido por sistemas externos, como APIs ou mecanismos de persistência.

Diferentemente das entidades do domínio, que representam conceitos do problema tratado pela aplicação, os modelos da camada de dados refletem a estrutura técnica dos dados utilizados na comunicação com outros sistemas. Isso inclui formatos de serialização, como JSON, e possíveis adaptações necessárias para converter tipos de dados.

Essa distinção é importante porque APIs externas ou mecanismos de persistência frequentemente utilizam estruturas de dados que não correspondem exatamente ao modelo conceitual do domínio da aplicação. Ao manter essas representações separadas, evita-se que detalhes técnicos relacionados à comunicação ou armazenamento contaminem o núcleo conceitual do sistema.

O Código 6 apresenta um exemplo de modelo de dados utilizado para representar a estrutura retornada por uma API externa.

Código 6: Modelo de dados utilizado para conversão entre JSON e objetos da aplicação

```
class ProductModel {
  final int id;
  final String name;
  final double price;

  ProductModel({
    required this.id,
    required this.name,
    required this.price,
  });

  factory ProductModel.fromJson(Map<String, dynamic> json) {
    return ProductModel(
      id: json['id'],
      name: json['name'],
      price: json['price'].toDouble(),
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'name': name,
      'price': price,
```

```
    };
  }
}
```

Fonte: Elaborado pelo autor.

> Entidades pertencem à camada de domínio e não devem depender de frameworks, bibliotecas de interface ou mecanismos de persistência.

Nesse exemplo, a classe `ProductModel` descreve como os dados de um produto são representados no formato JSON utilizado por uma API externa. Os métodos `fromJson` e `toJson` são responsáveis por realizar a conversão entre a representação textual utilizada na comunicação com serviços externos e os objetos manipulados pela aplicação. Essa separação entre entidades do domínio e modelos de dados permite que alterações no formato da API ou no mecanismo de persistência não afetem diretamente o modelo conceitual da aplicação.

> Entidades representam conceitos do domínio da aplicação.
> Modelos de dados (DTO) representam formatos utilizados na comunicação com APIs, bancos de dados ou outros sistemas externos.

5.2 Datasources

Os *datasources* são componentes responsáveis pela execução direta de operações de entrada e saída de dados. Eles encapsulam a lógica necessária para interagir com APIs, bancos de dados locais ou qualquer outro mecanismo de armazenamento.

Ao concentrar operações de acesso a dados em componentes específicos, a arquitetura evita que detalhes de infraestrutura se espalhem por diferentes partes do sistema. Essa separação facilita a manutenção do código e permite que diferentes fontes de dados sejam substituídas ou combinadas sem impactar outras camadas da aplicação.

O Código 7 apresenta um exemplo de *datasource* responsável por realizar a comunicação com um serviço externo que fornece informações sobre produtos.

Código 7: Datasource responsável pela obtenção de produtos a partir de uma API externa

```
class ProductRemoteDatasource {
  final HttpClient client;

  ProductRemoteDatasource(this.client);

  Future<List<ProductModel>> getProducts() async {
    final response = await client.get('/products');

    final list = response.data as List;

    return list
        .map((json) => ProductModel.fromJson(json))
        .toList();
  }
}
```

Fonte: Elaborado pelo autor.

Nesse exemplo, o *datasource* realiza uma requisição HTTP a um serviço externo responsável por fornecer informações sobre produtos. O resultado da chamada é recebido no formato JSON

e convertido em uma lista de objetos `ProductModel`. Esses modelos representam a estrutura de dados utilizada na comunicação com a API e serão posteriormente utilizados pela camada de repositório, que realizará a conversão desses modelos em entidades do domínio antes de disponibilizar os dados para as camadas superiores da aplicação.

> Datasources são responsáveis exclusivamente por operações de entrada e saída de dados, como chamadas HTTP ou acesso a banco de dados. Eles não devem conter regras de negócio nem lógica relacionada à interface da aplicação.

5.3 Implementação do repositório

A camada de dados também contém implementações concretas dos contratos de repositório definidos na camada de domínio. Essas implementações são responsáveis por coordenar chamadas aos *datasources* e converter os modelos de dados obtidos em entidades do domínio.

Essa conversão é necessária porque a camada de domínio não deve depender de estruturas utilizadas na comunicação com serviços externos. Modelos técnicos, como aqueles utilizados para representar dados em formato JSON ou em registros de banco de dados, pertencem à camada de dados e não devem ser utilizados diretamente pelas camadas superiores da aplicação.

Assim, a responsabilidade de transformar modelos técnicos em entidades conceituais pertence à camada de dados. O Código 8 apresenta uma implementação de repositório responsável por realizar essa conversão.

Código 8: Implementação de repositório responsável por converter modelos de dados em entidades do domínio

```
class ProductRepositoryImpl implements ProductRepository {
  final ProductRemoteDatasource remote;

  ProductRepositoryImpl(this.remote);

  @override
  Future<List<Product>> getAll() async {
    final models = await remote.getProducts();

    return models
        .map((m) => Product(
              id: m.id,
              name: m.name,
              price: m.price,
            ))
        .toList();
  }
}
```

Fonte: Elaborado pelo autor.

Nesse exemplo, o repositório solicita dados ao *datasource*, obtendo uma lista de modelos de dados que representam a estrutura retornada pela API externa. Em seguida, cada modelo é convertido em uma entidade do domínio. Essa transformação garante que as camadas superiores da aplicação continuem trabalhando exclusivamente com entidades conceituais, preservando a independência do domínio em relação a detalhes técnicos de comunicação ou persistência.

> O repositório atua como uma camada de tradução entre o domínio e as fontes de dados, convertendo modelos técnicos em entidades utilizadas pela aplicação.

5.4 Responsabilidades arquiteturais

A camada de dados possui responsabilidades bem definidas dentro da arquitetura.

- **Integração com fontes externas de dados**: realizar comunicação com APIs, bancos de dados ou outros serviços responsáveis por armazenar ou fornecer informações.
- **Conversão de formatos de dados**: transformar representações externas, como JSON ou registros de banco de dados, em estruturas utilizadas pela aplicação.
- **Implementação de contratos do domínio**: fornecer implementações concretas para os repositórios definidos na camada de domínio.

Ao concentrar essas responsabilidades em uma camada específica, a arquitetura garante que detalhes de infraestrutura permaneçam isolados, preservando a independência das camadas responsáveis pela interface e pelas regras de negócio.

6 Camada Core

A camada *core* reúne componentes de infraestrutura compartilhados por diferentes partes da aplicação. Diferentemente das demais camadas da arquitetura, que estão diretamente relacionadas ao domínio do problema ou à interface com o usuário, o *core* concentra funcionalidades genéricas necessárias para o funcionamento do sistema como um todo.

Esses componentes não representam conceitos do domínio e também não pertencem especificamente à interface ou ao acesso a dados. Em vez disso, fornecem serviços auxiliares que podem ser utilizados por múltiplas camadas da aplicação.

Em arquiteturas bem estruturadas, essa camada atua como um conjunto de utilidades reutilizáveis, reduzindo duplicação de código e promovendo maior padronização na implementação de funcionalidades técnicas.

Entre os elementos frequentemente presentes nessa camada estão mecanismos de tratamento de erros, clientes de comunicação HTTP, utilitários de logging, serviços de configuração e classes auxiliares utilizadas em diferentes partes do sistema.

6.1 Tratamento de erros

Em aplicações reais, diferentes operações podem falhar devido a diversos fatores, como indisponibilidade de serviços externos, falhas de rede ou inconsistências nos dados retornados por APIs. Para lidar com essas situações de forma consistente, é comum definir estruturas padronizadas de erro dentro da camada *core*.

Essas estruturas permitem representar falhas de maneira uniforme em toda a aplicação, facilitando a propagação e o tratamento de erros entre diferentes camadas do sistema. O Código 9 apresenta uma estrutura simples utilizada para representar erros dentro da aplicação.

Código 9: Estrutura utilizada para representar falhas na aplicação

```
class Failure implements Exception {
  final String message;

  Failure(this.message);
}
```

Fonte: Elaborado pelo autor.

Nesse exemplo, a classe `Failure` representa uma estrutura simples de erro utilizada para encapsular mensagens relacionadas a falhas ocorridas durante a execução de operações. Ao utilizar uma estrutura padronizada para representar erros, diferentes camadas da aplicação podem comunicar falhas de maneira consistente, independentemente da origem do problema.

> Erros técnicos (rede, banco de dados, API) devem ser convertidos em estruturas de falha padronizadas antes de serem propagados para outras camadas da aplicação.

6.2 Cliente HTTP

Outro componente frequentemente centralizado na camada *core* é o cliente responsável por realizar comunicação com serviços externos. Em vez de permitir que diferentes partes da aplicação realizem requisições HTTP diretamente, um cliente dedicado pode encapsular essa funcionalidade, oferecendo uma interface única para comunicação de rede.

Essa centralização facilita a aplicação de políticas comuns, como tratamento de erros, configuração de cabeçalhos de requisição, controle de autenticação ou registro de logs relacionados às operações de rede. O Código 10 apresenta um exemplo simplificado de cliente HTTP responsável por encapsular operações de comunicação com serviços externos.

Código 10: Cliente HTTP centralizado utilizado para comunicação com serviços externos

```
class HttpClient {
  Future<Response> get(String path) async {
    // implementa  o HTTP
  }
}
```

Fonte: Elaborado pelo autor.

Ao utilizar um cliente HTTP centralizado, os componentes responsáveis pelo acesso a dados podem concentrar-se exclusivamente na lógica de obtenção e transformação das informações. Dessa forma, detalhes técnicos relacionados à comunicação de rede permanecem encapsulados em um componente especializado, promovendo maior organização e reutilização dentro da arquitetura da aplicação.

> Clientes HTTP centralizados permitem padronizar comportamentos de rede em toda a aplicação, evitando duplicação de código e facilitando a manutenção da infraestrutura de comunicação.

6.3 Responsabilidades arquiteturais

A camada *core* possui um papel de suporte dentro da arquitetura da aplicação, oferecendo funcionalidades reutilizáveis que podem ser compartilhadas por diferentes camadas.

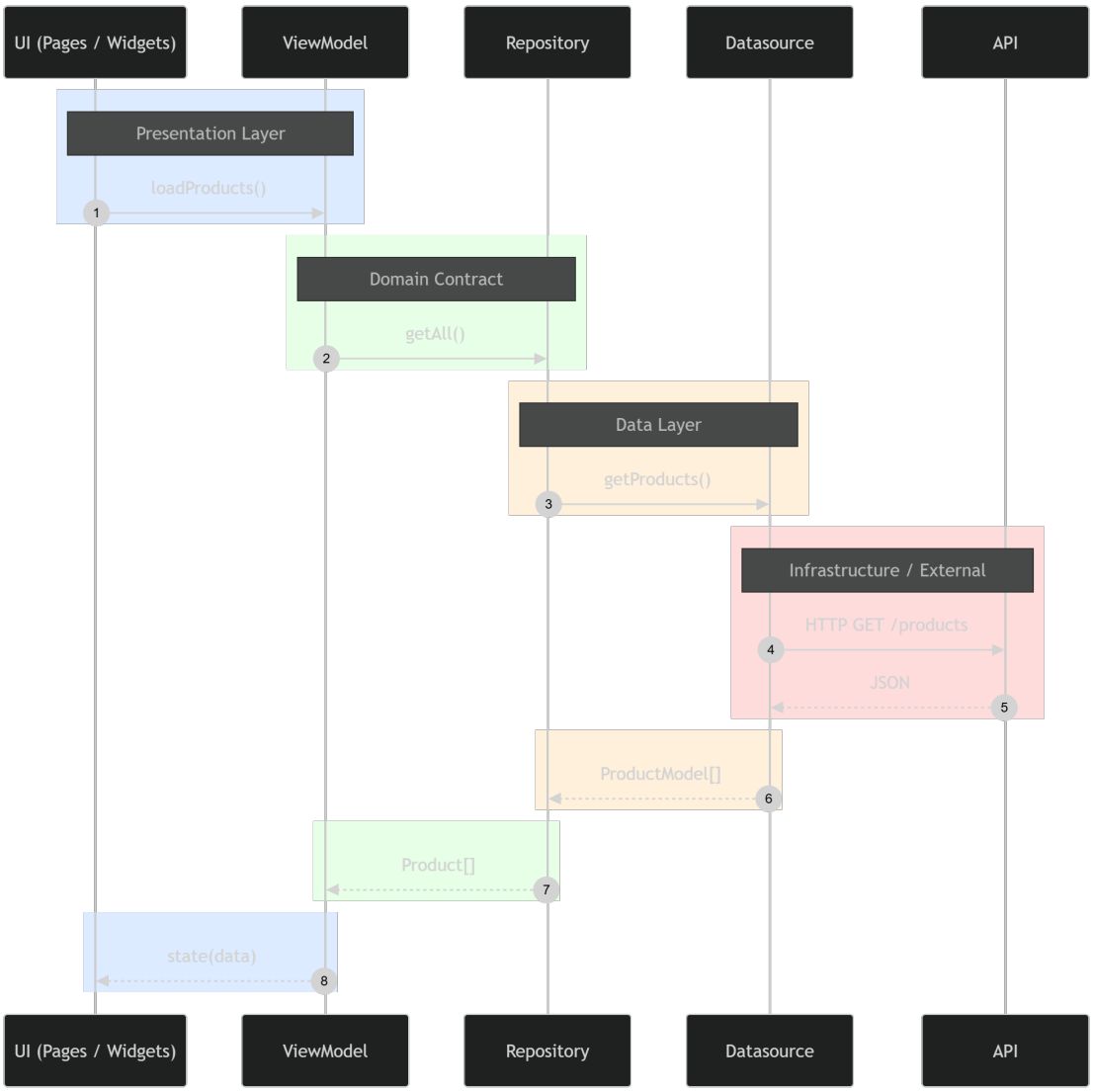
- **Infraestrutura compartilhada**: fornecer componentes técnicos utilizados por diversas partes da aplicação, como clientes de rede, utilitários e mecanismos de configuração.
- **Padronização de comportamentos**: estabelecer estruturas comuns para tratamento de erros, registro de logs ou manipulação de respostas de serviços externos.
- **Redução de duplicação**: evitar que funcionalidades técnicas sejam implementadas repetidamente em diferentes módulos do sistema.

Ao centralizar esses elementos em uma camada específica, a arquitetura mantém a separação entre lógica de negócio, interface do usuário e detalhes técnicos de infraestrutura, contribuindo para uma organização mais clara e sustentável do código ao longo do tempo.

7 Fluxo de Execução

O fluxo de execução da aplicação inicia-se na camada de apresentação, a partir de uma interação do usuário com a interface gráfica. Essa interação dispara uma ação no *ViewModel*, que é responsável por coordenar o estado da tela e orquestrar as operações necessárias para atender à solicitação do usuário. A Figura 2 apresenta esse fluxo do início ao fim.

Figura 2: Fluxo de execução entre as camadas



Fonte: Elaborado pelo autor.

Ao receber o evento da interface, o *ViewModel* invoca um método definido no contrato do *Repository*, localizado na camada de domínio. Nesse ponto ocorre uma separação importante de responsabilidades: a camada de apresentação conhece apenas o contrato do repositório, sem depender de detalhes de implementação, o que preserva o baixo acoplamento entre as camadas. A implementação concreta do repositório encontra-se na camada de dados. Essa implementação é responsável por delegar a operação para um *Datasource*, que representa o ponto de acesso real às fontes de dados, como APIs REST, bancos de dados locais ou outros serviços externos.

O *Datasource* realiza a comunicação com a API, normalmente por meio de requisições HTTP. A resposta retornada pela API, geralmente em formato JSON, é então convertida em objetos de modelo (*models* ou DTOs). Esses modelos representam a estrutura de dados utilizada para

comunicação com sistemas externos.

Após essa etapa, o repositório realiza o mapeamento entre os modelos da camada de dados e as entidades da camada de domínio. Esse processo garante que a lógica de negócio opere sobre estruturas independentes de detalhes de infraestrutura, preservando a pureza do domínio. As entidades resultantes são então retornadas ao *ViewModel*, que atualiza o estado da interface. A camada de apresentação observa essa mudança de estado e reconstrói a interface declarativamente, refletindo os novos dados para o usuário.

Esse fluxo estabelece um caminho claro de dependências, no qual as camadas externas dependem das internas, mas nunca o contrário. Dessa forma, a arquitetura promove separação de responsabilidades, maior testabilidade e facilidade de manutenção do sistema.

8 Dependência entre Camadas

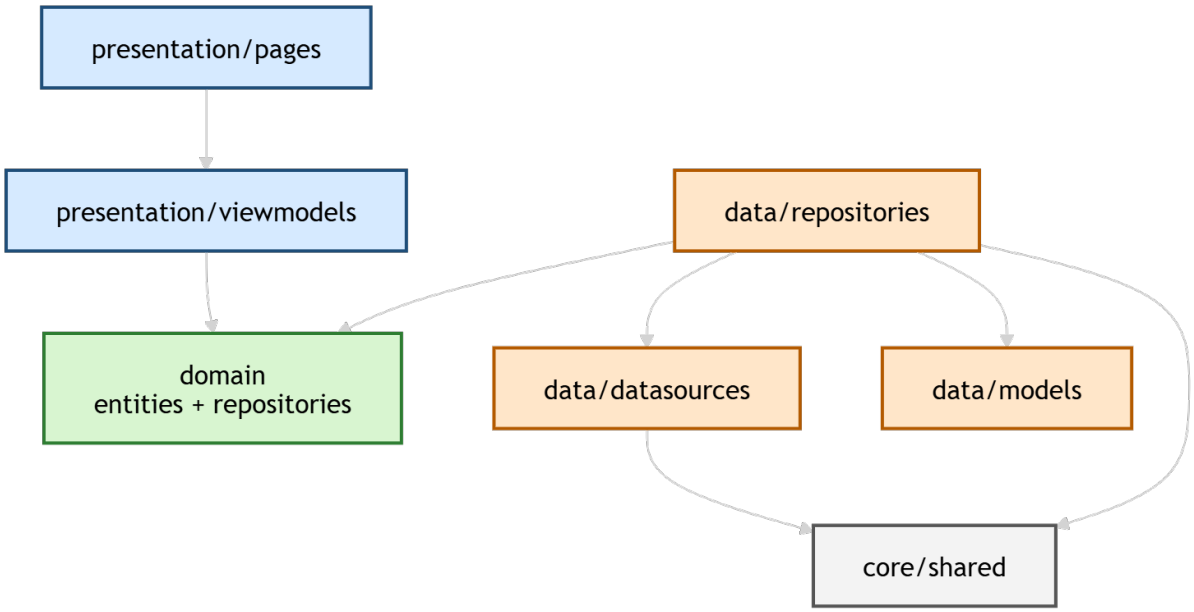
A arquitetura proposta estabelece regras claras de dependência entre as camadas do sistema. Essas regras têm como objetivo reduzir o acoplamento entre componentes, facilitar a manutenção do código e tornar a aplicação mais testável e evolutiva.

De forma geral, as camadas mais externas podem depender das camadas mais internas, porém o contrário nunca deve ocorrer. Isso significa que a lógica de domínio permanece independente de detalhes de infraestrutura, frameworks ou mecanismos de acesso a dados. A camada de *presentation* é responsável pela interface com o usuário e pela gestão do estado da interface. Para executar operações de negócio, essa camada depende apenas dos contratos definidos no domínio, normalmente por meio de interfaces de repositório. Dessa forma, a interface não precisa conhecer como os dados são obtidos ou persistidos.

A camada de *data*, por sua vez, contém as implementações concretas desses contratos. Ela é responsável por acessar fontes de dados externas, como APIs ou bancos de dados, além de realizar a conversão entre modelos de dados utilizados na comunicação externa e as entidades do domínio. No centro da arquitetura encontra-se a camada de *domain*. Essa camada contém as entidades e os contratos que representam as regras de negócio do sistema. Por ser independente de frameworks e tecnologias específicas, ela pode ser facilmente testada e reutilizada em diferentes contextos.

A Figura 3 apresenta visualmente essas relações de dependência entre as camadas.

Figura 3: Fluxo de dependências



Fonte: Elaborado pelo autor.

A regra fundamental de dependência pode ser resumida da seguinte forma:
presentation → domain
data → domain
domain → nenhuma camada

Essa organização garante que o domínio permaneça isolado de detalhes técnicos da aplicação, evitando que mudanças em APIs, frameworks ou bibliotecas externas impactem diretamente as regras de negócio.

9 Exercícios

9.1 Atividade Guiada: Construção de uma Aplicação Flutter com Arquitetura em Camadas

Nesta atividade será desenvolvido um pequeno aplicativo Flutter capaz de consumir uma API pública de produtos e exibir esses dados em uma interface simples. O objetivo principal da atividade não é apenas construir a interface, mas exercitar a separação arquitetural apresentada nas seções anteriores.

Ao final da atividade, espera-se que os estudantes sejam capazes de organizar um projeto Flutter seguindo uma arquitetura baseada em camadas, separando interface, lógica de fluxo, modelo de domínio e acesso a dados.

Objetivo da atividade

Desenvolver uma aplicação Flutter que:

- consuma uma API pública de produtos
- organize o código utilizando as camadas apresentadas anteriormente
- exiba uma lista de produtos em uma interface simples
- utilize boas práticas de separação de responsabilidades

A API utilizada será a *Fake Store API*, disponível publicamente no endereço:

```
https://fakestoreapi.com/products
```

Essa API retorna uma lista de produtos contendo informações como identificador, título, preço e imagem.

Etapa 1: Criação do projeto

Inicialmente, crie um novo projeto Flutter utilizando o terminal.

```
flutter create product_app
```

Após a criação do projeto, navegue até o diretório do aplicativo.

```
cd product_app
```

Em seguida, abra o projeto em seu ambiente de desenvolvimento.

Etapa 2: Organização da estrutura de pastas

Dentro do diretório `lib`, organize as pastas seguindo a arquitetura estudada.

```
lib/

  presentation/
    pages/
    viewmodels/

  domain/
    entities/
    repositories/

  data/
    models/
    datasources/
    repositories/

  core/
    network/
    errors/
```

Essa estrutura separa claramente os diferentes papéis arquiteturais do sistema.

Etapa 3: Criação da entidade de domínio

Crie o arquivo:

```
lib/domain/entities/product.dart
```

Nesse arquivo será definida a entidade principal do domínio.

```dart
class Product {
  final int id;
  final String title;
  final double price;
  final String image;

  const Product({
    required this.id,
    required this.title,
    required this.price,
    required this.image,
  });
}
```

Essa classe representa o conceito de produto dentro da aplicação.

Etapa 4: Definição do contrato de repositório

Crie o arquivo:

```
lib/domain/repositories/product_repository.dart
```

```dart
abstract class ProductRepository {
  Future<List<Product>> getProducts();
}
```

Esse contrato define quais operações o domínio precisa executar para obter os dados.

Etapa 5: Modelos de dados

Crie o arquivo:

```
lib/data/models/product_model.dart
```

Esse modelo representa o formato de dados retornado pela API.

```dart
class ProductModel {
  final int id;
  final String title;
  final double price;
  final String image;

  ProductModel({
    required this.id,
    required this.title,
    required this.price,
    required this.image,
  });

  factory ProductModel.fromJson(Map<String, dynamic> json) {
    return ProductModel(
      id: json["id"],
      title: json["title"],
      price: json["price"].toDouble(),
      image: json["image"],
    );
  }
}
```

Etapa 6: Criação do datasource

Crie o arquivo:

```
lib/data/datasources/product_remote_datasource.dart
```

Esse componente será responsável por realizar a requisição HTTP.

```dart
class ProductRemoteDatasource {
  final HttpClient client;

  ProductRemoteDatasource(this.client);

  Future<List<ProductModel>> getProducts() async {
    final response = await client.get(
      "https://fakestoreapi.com/products"
    );
    final List data = response.data;
    return data
      .map((json) => ProductModel.fromJson(json))
      .toList();
  }
}
```

Etapa 7: Implementação do repositório

Crie o arquivo:

```
lib/data/repositories/product_repository_impl.dart
```

```dart
class ProductRepositoryImpl implements ProductRepository {

  final ProductRemoteDatasource datasource;

  ProductRepositoryImpl(this.datasource);

  @override
  Future<List<Product>> getProducts() async {

    final models = await datasource.getProducts();

    return models
      .map((m) => Product(
        id: m.id,
        title: m.title,
        price: m.price,
        image: m.image,
      ))
      .toList();
  }
}
```

Etapa 8: Criação do ViewModel

Crie o arquivo:

```
lib/presentation/viewmodels/product_viewmodel.dart
```

```dart
class ProductViewModel {
  final ProductRepository repository;

  final ValueNotifier<List<Product>> products =
      ValueNotifier([]);

  ProductViewModel(this.repository);

  Future<void> loadProducts() async {
    final result = await repository.getProducts();
    products.value = result;
  }
}
```

Etapa 9: Criação da interface

Crie o arquivo:

```
lib/presentation/pages/product_page.dart
```

A página será responsável apenas por exibir os produtos.

```
class ProductPage extends StatelessWidget {
  final ProductViewModel viewModel;
  const ProductPage({super.key, required this.viewModel});

  @override
  Widget build(BuildContext context) {

    return Scaffold(
      appBar: AppBar(title: const Text("Products")),

      body: ValueListenableBuilder<List<Product>>(

        valueListenable: viewModel.products,

        builder: (context, products, _) {

          return ListView.builder(
            itemCount: products.length,

            itemBuilder: (context, index) {

              final product = products[index];

              return ListTile(
                leading: Image.network(product.image),
                title: Text(product.title),
                subtitle: Text("\$${product.price}"),
              );
            },
          );
        },
      ),

      floatingActionButton: FloatingActionButton(
        onPressed: viewModel.loadProducts,
        child: const Icon(Icons.download),
      ),
    );
  }
}
```

Resultado esperado

Ao concluir a atividade, a aplicação deverá ser capaz de:

- realizar uma requisição HTTP para a API de produtos
- converter os dados recebidos para objetos da aplicação
- atualizar o estado do ViewModel
- exibir uma lista de produtos na interface

Além disso, o projeto deverá manter a separação entre as camadas *presentation*, *domain*, *data* e *core*, preservando a organização arquitetural proposta neste material.

9.2 Atividade Guiada 2: Refatoração Arquitetural

Após a implementação inicial da aplicação, é comum perceber que algumas funcionalidades essenciais ainda não estão presentes. Sistemas reais precisam lidar com situações como falhas de comunicação, estados intermediários da interface e estratégias para reduzir dependência de rede.

Nesta atividade, o objetivo é evoluir a aplicação construída anteriormente adicionando três elementos fundamentais em aplicações modernas:

- tratamento de erros
- estado de carregamento da interface
- mecanismo simples de cache local

Essas melhorias permitem compreender como a arquitetura em camadas facilita a evolução do sistema sem comprometer sua organização estrutural.

Objetivo da atividade

Refatorar o projeto desenvolvido na atividade anterior para que ele seja capaz de:

- informar ao usuário quando os dados estão sendo carregados
- tratar falhas de comunicação com a API
- utilizar dados previamente carregados quando não houver conexão

Etapa 1: Modelagem do estado da interface

Inicialmente, será necessário representar explicitamente o estado da interface. Para isso, criaremos uma estrutura capaz de indicar se a aplicação está carregando dados, se ocorreu um erro ou se os produtos foram obtidos com sucesso.

Crie o arquivo:

```
lib/presentation/viewmodels/product_state.dart
```

```dart
class ProductState {
  final bool isLoading;
  final List<Product> products;
  final String? error;

  const ProductState({
    this.isLoading = false,
    this.products = const [],
    this.error,
  });

  ProductState copyWith({
    bool? isLoading,
    List<Product>? products,
    String? error,
  }) {
    return ProductState(
      isLoading: isLoading ?? this.isLoading,
      products: products ?? this.products,
```

```
      error: error,
    );
  }
}
```

Essa estrutura permite representar diferentes estados da aplicação, como carregamento, sucesso ou falha.

Etapa 2: Atualização do ViewModel

Agora o *ViewModel* será responsável por atualizar o estado da tela conforme o resultado das operações.

Modifique o arquivo:

```
lib/presentation/viewmodels/product_viewmodel.dart
```

```
class ProductViewModel {
  final ProductRepository repository;

  final ValueNotifier<ProductState> state =
      ValueNotifier(const ProductState());

  ProductViewModel(this.repository);

  Future<void> loadProducts() async {
    state.value = state.value.copyWith(isLoading: true);
    try {
      final products = await repository.getProducts();
      state.value = state.value.copyWith(
        isLoading: false,
        products: products,
      );
    } catch (e) {
      state.value = state.value.copyWith(
        isLoading: false,
        error: e.toString(),
      );
    }
  }
}
```

Agora o *ViewModel* controla explicitamente o estado da aplicação.

Etapa 3: Atualização da interface

A interface deve reagir ao estado atual da aplicação.

Atualize o arquivo:

```
lib/presentation/pages/product_page.dart
```

```
body: ValueListenableBuilder<ProductState>(
  valueListenable: viewModel.state,

  builder: (context, state, _) {
    if (state.isLoading) {
      return const Center(
```

```
        child: CircularProgressIndicator(),
      );
    }
    if (state.error != null) {
      return Center(
        child: Text(state.error!),
      );
    }
    return ListView.builder(
      itemCount: state.products.length,
      itemBuilder: (context, index) {
        final product = state.products[index];
        return ListTile(
          title: Text(product.title),
          subtitle: Text("\$${product.price}"),
        );
      },
    );
  },
),
```

Agora a interface reage dinamicamente ao estado produzido pelo *ViewModel*.

Etapa 4: Tratamento de erros na camada Core

Para padronizar o tratamento de erros, será utilizada uma estrutura definida na camada *core*.

Crie o arquivo:

```
lib/core/errors/failure.dart
```

```
class Failure implements Exception {
  final String message;

  Failure(this.message);
}
```

Esse tipo de estrutura permite representar falhas de maneira consistente em toda a aplicação.

Etapa 5: Implementação de cache local

Uma forma simples de melhorar a experiência do usuário é armazenar os dados carregados anteriormente. Caso a API esteja indisponível, o sistema pode utilizar os dados armazenados.

Crie o arquivo:

```
lib/data/datasources/product_cache_datasource.dart
```

```
class ProductCacheDatasource {
  List<ProductModel>? _cache;

  void save(List<ProductModel> products) {
    _cache = products;
  }
  List<ProductModel>? get() {
    return _cache;
  }
}
```

Esse cache simples funciona apenas enquanto a aplicação estiver em execução.

Etapa 6: Atualização do repositório

O repositório agora poderá utilizar tanto o datasource remoto quanto o cache local.

Modifique o arquivo:

```
lib/data/repositories/product_repository_impl.dart
```

```
class ProductRepositoryImpl implements ProductRepository {
  final ProductRemoteDatasource remote;
  final ProductCacheDatasource cache;

  ProductRepositoryImpl(this.remote, this.cache);

  @override
  Future<List<Product>> getProducts() async {

    try {
      final models = await remote.getProducts();

      cache.save(models);

      return models
        .map((m) => Product(
          id: m.id,
          title: m.title,
          price: m.price,
          image: m.image,
        ))
        .toList();
    } catch (e) {
      final cached = cache.get();
      if (cached != null) {
        return cached
          .map((m) => Product(
            id: m.id,
            title: m.title,
            price: m.price,
            image: m.image,
          ))
          .toList();
      }
      throw Failure("N o foi poss vel carregar os produtos");
    }
  }
}
```

Agora o repositório possui uma estratégia simples de *fallback* para cache.

Resultado esperado

Após concluir a atividade, a aplicação deverá apresentar três melhorias importantes:

- um indicador visual de carregamento enquanto os dados são obtidos
- tratamento adequado de falhas na comunicação com a API

- reutilização de dados previamente carregados quando a API não estiver disponível

Essas melhorias demonstram como a separação arquitetural facilita a evolução do sistema sem exigir mudanças significativas na estrutura da aplicação.

Reflexão

Após concluir a atividade, discuta as seguintes questões:

- Em qual camada foi implementado o cache e por quê?
- Por que o *ViewModel* não realiza chamadas HTTP diretamente?
- O que aconteceria se a interface acessasse diretamente o datasource?
- Como essa arquitetura facilitaria a substituição da API por um banco de dados local?

Responder a essas perguntas ajuda a compreender o papel de cada camada dentro da arquitetura do sistema.

Sobre este material

Este material foi desenvolvido por:

Jefferson Rodrigo Speck
Professor e pesquisador na área de Computação.

O conteúdo foi produzido com finalidade educacional e pode ser utilizado como material de apoio em disciplinas relacionadas à área.

Versão compilada em 4 de março de 2026